

MANIPAL
ACADEMY of HIGHER EDUCATION
(Institution of Eminence Deemed to be University)

MANIPAL SCHOOL OF INFORMATION SCIENCES

(A Constituent unit of MAHE, Manipal)

CYE 5103 Linux OS and Scripting – 4

Reg. Number	Name	Branch
261100690010	Hegde Aansh Dilip	Cyber Security

11/09/2026



MANIPAL SCHOOL OF INFORMATION SCIENCES
MANIPAL
(A constituent unit of MAHE, Manipal)

Hard link vs. soft link

Part A — Create the links

Create a directory named link_lab and navigate to it.

Create a file named security.txt containing the following text:

Linux File System Security Lab

Confidential Security Report

Create:

a hard link named security_hard.txt

a symbolic link named security_soft.txt

List all three files using a command that displays their inode numbers.

Record the inode numbers of:

security.txt

security_hard.txt

security_soft.txt

```
┌──(kali㉿kali)-[~]
└─$ mkdir link_lab

┌──(kali㉿kali)-[~]
└─$ cd link_lab

┌──(kali㉿kali)-[~/link_lab]
└─$ echo -e "Linux File System Security Lab\nConfidential Security Report" > security.txt

┌──(kali㉿kali)-[~/link_lab]
└─$ ln security.txt security_hard.txt

┌──(kali㉿kali)-[~/link_lab]
└─$ ln -s security.txt security_soft.txt

┌──(kali㉿kali)-[~/link_lab]
└─$ ls -li
total 8
1179901 -rw-rw-r-- 2 kali kali 60 Sep 11 08:13 security_hard.txt
1179902 lrwxrwxrwx 1 kali kali 12 Sep 11 08:14 security_soft.txt → security.txt
1179901 -rw-rw-r-- 2 kali kali 60 Sep 11 08:13 security.txt

┌──(kali㉿kali)-[~/link_lab]
└─$ cat security.txt
Linux File System Security Lab
Confidential Security Report
```

What difference do you observe?

From the output, security.txt and security_hard.txt are basically referring to the same file, so they show the same inode number.

However, security_soft.txt is different because it is a shortcut (symbolic link) to security.txt, so it has its own inode number.

Part B — Examine File Metadata

Use ls -l to examine the three files. Identify the file type of each file from the first character of the permission field.

Compare the link count of the original file, hard link, and symbolic link. Explain the observed values.

```
┌──(kali㉿kali)-[~/link_lab]
└─$ ls -l
total 8
-rw-rw-r-- 2 kali kali 60 Sep 11 08:13 security_hard.txt
lrwxrwxrwx 1 kali kali 12 Sep 11 08:14 security_soft.txt → security.txt
-rw-rw-r-- 2 kali kali 60 Sep 11 08:13 security.txt

┌──(kali㉿kali)-[~/link_lab]
└─$ stat security.txt
  File: security.txt
  Size: 60              Blocks: 8          IO Block: 4096   regular file
Device: 8,1     Inode: 1179901     Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/    kali)   Gid: ( 1000/    kali)
Access: 2026-09-11 08:14:28.798731619 -0400
Modify: 2026-09-11 08:13:17.375038786 -0400
Change: 2026-09-11 08:13:35.516104487 -0400
 Birth: 2026-09-11 08:13:17.375038786 -0400

┌──(kali㉿kali)-[~/link_lab]
└─$ stat security_hard.txt
  File: security_hard.txt
  Size: 60              Blocks: 8          IO Block: 4096   regular file
Device: 8,1     Inode: 1179901     Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/    kali)   Gid: ( 1000/    kali)
Access: 2026-09-11 08:14:28.798731619 -0400
Modify: 2026-09-11 08:13:17.375038786 -0400
Change: 2026-09-11 08:13:35.516104487 -0400
 Birth: 2026-09-11 08:13:17.375038786 -0400

┌──(kali㉿kali)-[~/link_lab]
└─$ stat security_soft.txt
  File: security_soft.txt → security.txt
  Size: 12              Blocks: 0          IO Block: 4096   symbolic link
Device: 8,1     Inode: 1179902     Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/    kali)   Gid: ( 1000/    kali)
Access: 2026-09-11 08:14:10.705689907 -0400
Modify: 2026-09-11 08:14:03.105892042 -0400
Change: 2026-09-11 08:14:03.105892042 -0400
 Birth: 2026-09-11 08:14:03.105892042 -0400

┌──(kali㉿kali)-[~/link_lab]
```

Use the stat command on all three files and record the following information:

Parameter	Original	Hard Link	Soft Link
Inode number	1179901	1179901	1179902
File type	Regular File	Regular File	Symbolic link
Size	60 bytes	60 bytes	12 bytes
Link count	2	2	1
Permissions	-rw-rw-r--	-rw-rw-r--	lrwxrwxrwx

Compare the file sizes of the original file, hard link, and symbolic link. Why might the size of the symbolic link differ from the original file?

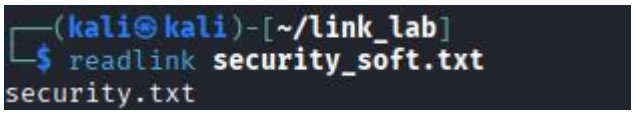
The original file and the hard link both show 60 bytes because they refer to the same actual data. The soft link shows only 12 bytes because it contains the reference/name of the original file instead of storing the file's complete content.

File type — first character of permission field

File	First char	Type
security.txt	-	Regular file
security_hard.txt	-	Regular file
security_soft.txt	1	Symbolic link

Part C — Investigate readlink

1. Execute readlink on security_soft.txt. What information is displayed?

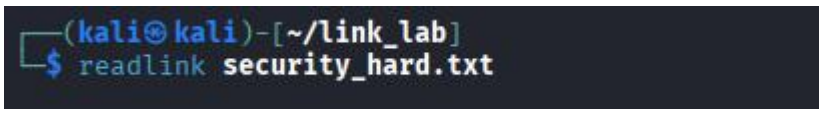


readlink security_soft.txt

The output is: security.txt

This shows that security_soft.txt is a symbolic link pointing to security.txt.

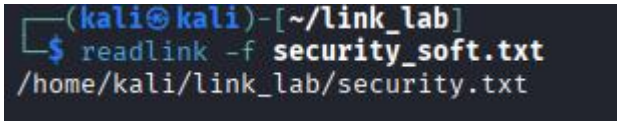
2.Execute readlink on security_hard.txt. What do you observe?



Observation:

There is no output because security_hard.txt is a hard link. A hard link does not store the pathname of the original file; it directly refers to the same inode.

3.Based on your observations, explain why readlink works differently for hard links and symbolic links.



It displays the complete absolute path of the final target of the symbolic link, for example: /home/kali/link_lab/security.txt

Explanation:

The soft link gives the name of the file it points to, while the hard link gives no output because it directly refers to the same inode. The -f option gives the complete absolute path of the original file.

4. What does the -f option reveal?

The -f option reveals the complete absolute path of the final target of the symbolic link.



Part D — Content Modification

1.Append the following line to security.txt: Unauthorized login detected

This appends the line to security.txt without deleting the existing content.

2.Display the contents using all three filenames.

cat security.txt

cat security_hard.txt

cat security_soft.txt

The line Unauthorized login detected appears in all three outputs, because the hard link and symbolic link both access the same file content.

3.Is the newly added content visible through both links? Explain why

After adding new content to security.txt, the same updated content can be seen through both security_hard.txt and security_soft.txt. This is because the hard link refers to the same inode as the original file, while the soft link points to the original file. Therefore, changes made to the original file are reflected when accessed through both links.





Part E — Rename Experiment

security.txt

to:

incident.txt

1.Try accessing the content through security_hard.txt. Does it still work?

mv security.txt incident.txt

2.Try accessing the content through security_soft.txt. Does it still work?

cat security_hard.txt

Yes. The content is still accessible through the hard link.

3.Use readlink security_soft.txt after renaming the original file. What path is still stored in the symbolic link?

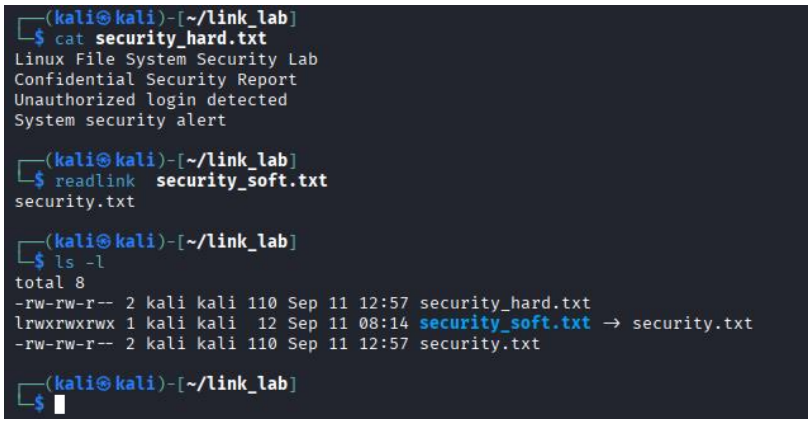
It will display: security.txt

This is because the symbolic link stores the original pathname, even though the file was renamed to incident.txt.

4.Explain why renaming the original file affects the symbolic link differently from the hard link.

Hard link: It points directly to the same inode, so renaming the original filename does not affect the hard link. It continues to work.

Symbolic link: It stores the pathname of the target (security.txt). When the original file is renamed to incident.txt, that pathname no longer exists, so the symbolic link becomes broken.



Part F — Deletion Experiment

1.Recreate the links if necessary and then perform the following.

security_hard.txt

2.Delete the original file.

rm security.txt

3.Check whether security_hard.txt can still be accessed.

cat security_hard.txt

Yes, security_hard.txt can still be accessed and will display the file contents. This is because the hard link refers directly to the same inode, even after the original filename is deleted.

4.Check whether security_soft.txt can still be accessed.

cat security_soft.txt

No, security_soft.txt cannot be accessed. It will give a “No such file or directory” error because its target security.txt has been deleted. The symbolic link becomes a dangling/broken link.

5.Use ls -l and readlink to investigate the symbolic link after deleting its target.

ls -l security_soft.txt

readlink security_soft.txt

ls -l shows:

security_soft.txt -> security.txt

This shows the symbolic link still exists, but its target has been deleted. readlink displays: security.txt This confirms that the link still stores the old target path. security_soft.txt is now a dangling/broken symbolic link.

6.What is a dangling/broken symbolic link?

A dangling/broken symbolic link is a symbolic link whose target file or path no longer exists.

7.Does deleting the original filename immediately delete the underlying data when a hard link exists? Justify your answer using the link count and inode.

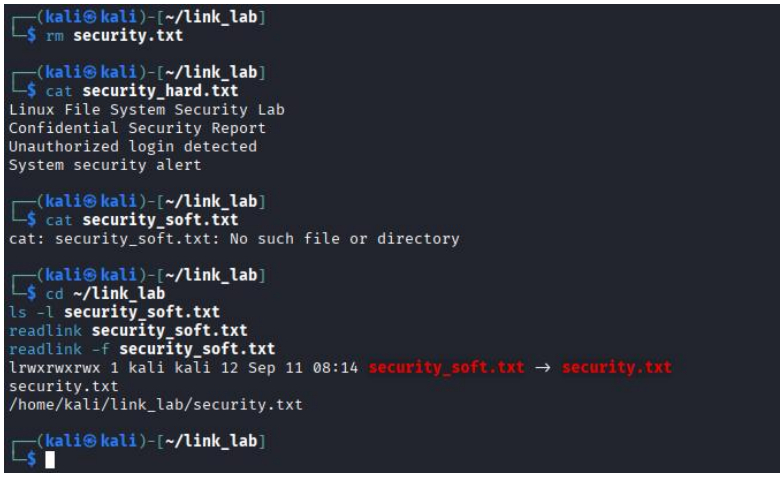
No. Deleting the original filename does not immediately delete the underlying data when a hard link exists.

Before deletion, the inode has a link count of 2 (security.txt and security_hard.txt).

After deleting security.txt, the link count becomes 1.

The same inode and underlying data remain, so security_hard.txt can still access the contents.

The data is removed only when the link count reaches 0 (and there are no open references).



Part G — Directory and Filesystem Behaviour

1.Create a directory named evidence. Attempt to create a hard link to this directory. Record the result.

mkdir evidence

ln evidence evidence_hard

Expected result:

ln: evidence: hard link not allowed for directory

The hard link creation fails because hard links to directories are not normally allowed.

2.Create a symbolic link named evidence_link pointing to the evidence directory. Was the operation successful?

Yes, the operation is successful.

ln -s evidence evidence_link

The symbolic link evidence_link is successfully created and points to the evidence directory. Symbolic links can point to directories.

3.Based on your experiment, compare hard and symbolic links with respect to directory linking.

Hard link: Cannot normally be created for a directory. The attempt fails.

Symbolic link: Can be created for a directory successfully. evidence_link points to the evidence directory.

If multiple filesystems/mount points are available in the laboratory environment, attempt to create:

a hard link across filesystems

A hard link cannot be created across different filesystems. The operation should fail with an error such as: Invalid cross-device link

a symbolic link across filesystems

A symbolic link can be created across different filesystems: ln -s /path/on/other/filesystem evidence_link

Hard links must be on the same filesystem because they refer to the same inode. Symbolic links can cross filesystems because they store the target pathname.

Final Comparison

Parameter	**Hard Link**	**Symbolic Link**
Inode number	Same as target	Different from target
File type	Regular file	Symbolic link
Link count	Increases	Usually 1
File size	Same as target	Size of stored pathname
Stores target pathname	No	Yes
readlink output	No output	Shows target pathname

Effect of modifying target	Change is visible	Change is visible
Effect of renaming target	Still works	Becomes broken
Effect of deleting target	Still works if another hard link exists	Becomes broken
Can become a dangling or broken link?	No	Yes
Directory linking	Not normally allowed	Allowed
Cross-filesystem linking	Not allowed	Allowed

sort

Suppose alerts.txt contains:

192.168.1.10 admin 5

10.0.0.8 root 12

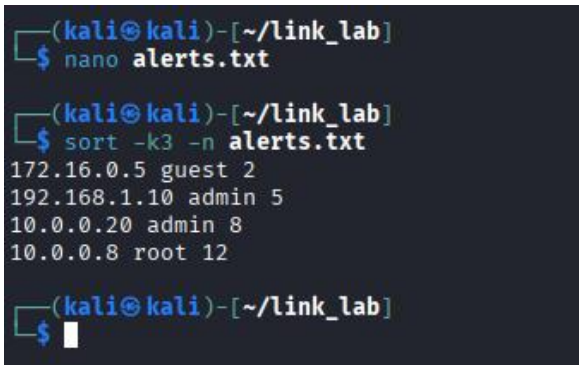
172.16.0.5 guest 2

10.0.0.20 admin 8

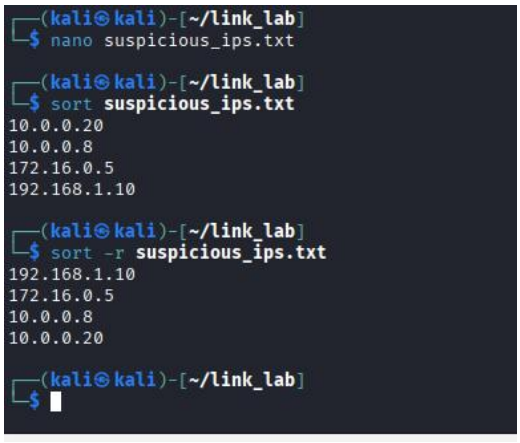
Format:

IP-address username failed attempts

Sort according to failed attempts



Sort a list of suspicious IP addresses alphabetically.



Display only unique suspicious IP addresses using sort.

```
┌──(kali㉿kali)-[~/link_lab]
└─$ sort -u suspicious_ips.txt
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10

┌──(kali㉿kali)-[~/link_lab]
└─$
```

Sort a list of detected port numbers numerically.

```
┌──(kali㉿kali)-[~/link_lab]
└─$ nano ports.txt

┌──(kali㉿kali)-[~/link_lab]
└─$ sort -n ports.txt
10
21
80
443
8080

┌──(kali㉿kali)-[~/link_lab]
└─$ sort -nr ports.txt
8080
443
80
21
10

┌──(kali㉿kali)-[~/link_lab]
└─$
```

Display port numbers in descending numerical order.

```
┌──(kali㉿kali)-[~/link_lab]
└─$ sort -nr ports.txt
8080
443
80
21
10
```

Sort usernames without considering uppercase/lowercase differences.

```
┌──(kali㉿kali)-[~/link_lab]
└─$ nano usernames.txt

┌──(kali㉿kali)-[~/link_lab]
└─$ sort -f usernames.txt
alice
Alice
bob
BOB
charlie
CHARLIE

┌──(kali㉿kali)-[~/link_lab]
└─$
```

From a failed-login log, count how many times each IP address occurs and display the most frequent IP first.

```
┌──(kali㉿kali)-[~/link_lab]
└─$ nano failed_login.log

┌──(kali㉿kali)-[~/link_lab]
└─$ sort failed_login.log | uniq -c | sort -nr
      3 10.0.0.20
      2 10.0.0.8
      1 172.16.0.5
      1

┌──(kali㉿kali)-[~/link_lab]
└─$
```

Given IP,username,failed_attempts, sort records based on the number of failed attempts.

```
┌──(kali㉿kali)-[~/link_lab]
└─$ nano records.csv

┌──(kali㉿kali)-[~/link_lab]
└─$ sort -t ',' -k3,3n records.csv
192.168.1.10,admin,5
10.0.0.8,root,12
10.0.0.20,guest,22
10.0.0.20,admin,80

┌──(kali㉿kali)-[~/link_lab]
└─$
```

Sort a CSV vulnerability report based on severity score.

```
┌──(kali㉿kali)-[~/link_lab]
└─$ nano vulnerabilities.csv

┌──(kali㉿kali)-[~/link_lab]
└─$ sort -t ',' -k3,3n vulnerabilities.csv
CVE-004,Path Traversal,4
CVE-002,XSS,6
CVE-001,SQL Injection,9
CVE-003,Command Injection,10

┌──(kali㉿kali)-[~/link_lab]
└─$
```

Extract IP addresses using grep, then use sort and uniq to identify unique IP addresses.

```
┌──(kali㉿kali)-[~/link_lab]
└─$ grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' failed_login.log | sort -u
10.0.0.20
10.0.0.8
172.16.0.5
```

Create a pipeline that displays the top 5 IP addresses responsible for the most failed login attempts.

```
┌──(kali㉿kali)-[~/link_lab]
└─$ grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' failed_login.log | sort | uniq -c | sort -nr | head -5
      3 10.0.0.20
      2 10.0.0.8
      1 172.16.0.5
```